



THINKST
SCAPES

Q4 • 2024

<https://thinkst.com/ts>

Brought to you by



**Most companies find out way too late
that they've been breached.**

Thinkst Canary changes this.

Canaries deploy in under 4 minutes
and require 0 ongoing admin overhead.

They remain silent until they need to chirp,
and then, you receive that single alert.

When.it.matters.

Find out why some of the smartest security teams
in the world swear by Thinkst Canary.

<https://canary.love>

Contents

Introduction	2
Themes covered in this issue	3
A year in review	4
Wins and losses in the Microsoft ecosystem	5
Pointer Problems - Why We're Refactoring the Windows Kernel	6
Defending off the land	7
Unveiling the Power of Intune: Leveraging Intune for Breaking Into Your Cloud and On-Premise	8
From Simulation to Tenant Takeover	9
From Convenience to Contagion: The Libarchive Vulnerabilities Lurking in Windows 11	10
LLM hype continues, as do the security issues	11
Things we learned about LLMs in 2024	12
AI Meets Git: Unmasking Security Flaws in Qodo Merge	3
Suicide Bot: New AI Attack Causes LLM to Provide Potential "Self-Harm" Instructions	4
Diving deep, then diving deeper	15
Breaking NATO Radio Encryption	16
Exploiting File Writes in Hardened Environments	17
Hacking yourself a satellite - recovering BEESAT-1	18
IRIS: Non-Destructive Inspection of Silicon	19
SQL Injection Isn't Dead	20
Nifty sundries	21
What Developers Get for Free?	22
Dialing into the Past: RCE via the Fax Machine - Because Why Not?	23
Broken isolation - Draining your Credentials from Popular macOS Password Managers	24
I'll Be There for You! Perpetual Availability in the A^8 MVX System	25
Exploring and Exploiting an Android "Smart POS" Payment Terminal	26
Conclusions	27

Cover photo: The Baptistery of St. John the Baptist, Florence. Image by Casey Smith (Thinkst)

Introduction

Muscat, Oman. Image by Dev Dua (Thinkst)

Welcome to this ThinkstScapes edition, the 2024 wrap-up! No... We aren't pathologically tardy (or a year late!). This issue focuses on content released, published or presented between the first of October and the end of December, 2024.

We also reflect on 2024, comparing the themes and trends we extracted to see if we correctly identified the big movements in the industry. Q4 showed a slight downtick in the number of venues from Q3, but some very strong content was released right through the end of the year (i.e., even on 31 December!). Regardless of venue, there's some great material to cover this quarter!

As a reminder: if you are aware of work we've missed, a blog post we should have seen or a conference we should have covered, we'd love to hear about it. Please send them to ts@thinkst.com!

In addition to almost 1,400 blog posts, this quarter's content was drawn from talks and papers presented at the following conferences:

Conference Name	Number of talks
Black Hat EU	48
Hexacon	15
0xCon	12
BlueHat	32
BSides London	70
38C3	133
HushCon	16
SecTor	47
Virus Bulletin	72
ACM CCS 2024	76
ACSAC	94
Hack.lu / CTI Summit	98
No Hat Con	18
Hacktivity	26
SaintCon	77
Black Alps	17
Hardwear.io NL	37
HCKSYD24	20
POC	17
Code Blue	39
DeepSec	52
BSides Cape Town	17
CHCon	33
BSides Munich	28
CarolinaCon	6
OBTS	22
CornCon X	47
Total	1,169



Themes covered in this issue

Florence. Photo by Casey Smith (Thinkst)

WINS AND LOSSES IN THE MICROSOFT ECOSYSTEM

This theme highlights how expansive the Microsoft ecosystem is and the security challenges therein. From deep in the Windows OS all the way to cloud-based attack simulation, researchers have been working to improve user security or find flaws in the ecosystem. Look out for OS hardening efforts, both by Microsoft and system administrators; abusing Intune as an attacker; tenant takeover just from trying to use documented features; and how Open Source components used by Windows can become out-of-sync with upstream when it comes to security issues.

LLM HYPE CONTINUES, AS DO THE SECURITY ISSUES

Companies the world over have been trying to apply generative AI to their problem domains and a bunch of researchers followed suit. In this theme we pick one summary of how the LLM space has evolved, and two case studies on how LLMs can themselves harbor security issues.

DIVING DEEP, THEN DIVING DEEPER

Works in this theme shared the common aspect of depth. The researchers were curious and reframed research to be more general, despite that often making the work harder. Look for: a practical cryptography break in fielded military radios, exploiting arbitrary file writes on a read-only filesystem and hacking a satellite to recover it, a non-destructive way to verify hardware, and a database query smuggling technique.

NIFTY SUNDRIES

We found more works that we thought were stand-out, but didn't fit into this quarter's themes. Read on for: reviewing languages and their frameworks as a community security journey, hacking fax machines to run Doom, dumping password managers on MacOS, multi-variant execution, and hacking point-of-sale systems.

A year in review

We annually reflect on the signals we've extracted from our enormous selection of talks, blogs and papers (available on [Citation](#)). With this issue wrapping up 2024, a year that continued the security industry's turbulence of budget cuts and high-profile breaches from 2023, we wanted to highlight some of the research community's outstanding work. In the last 12 months, we extracted the following themes:



Q1:

- Revealing more than anticipated
- Taking another look with a fresh perspective
- Turning Windows into doors



Q2:

- AI/ML (in)security
- Whole system analysis
- New modalities of attack
- Old components showing the strain



Q3:

- Edge cases at internet-scale having big impacts
- Going above and beyond
- What goes on beyond the curtain can still be dangerous



Q4:

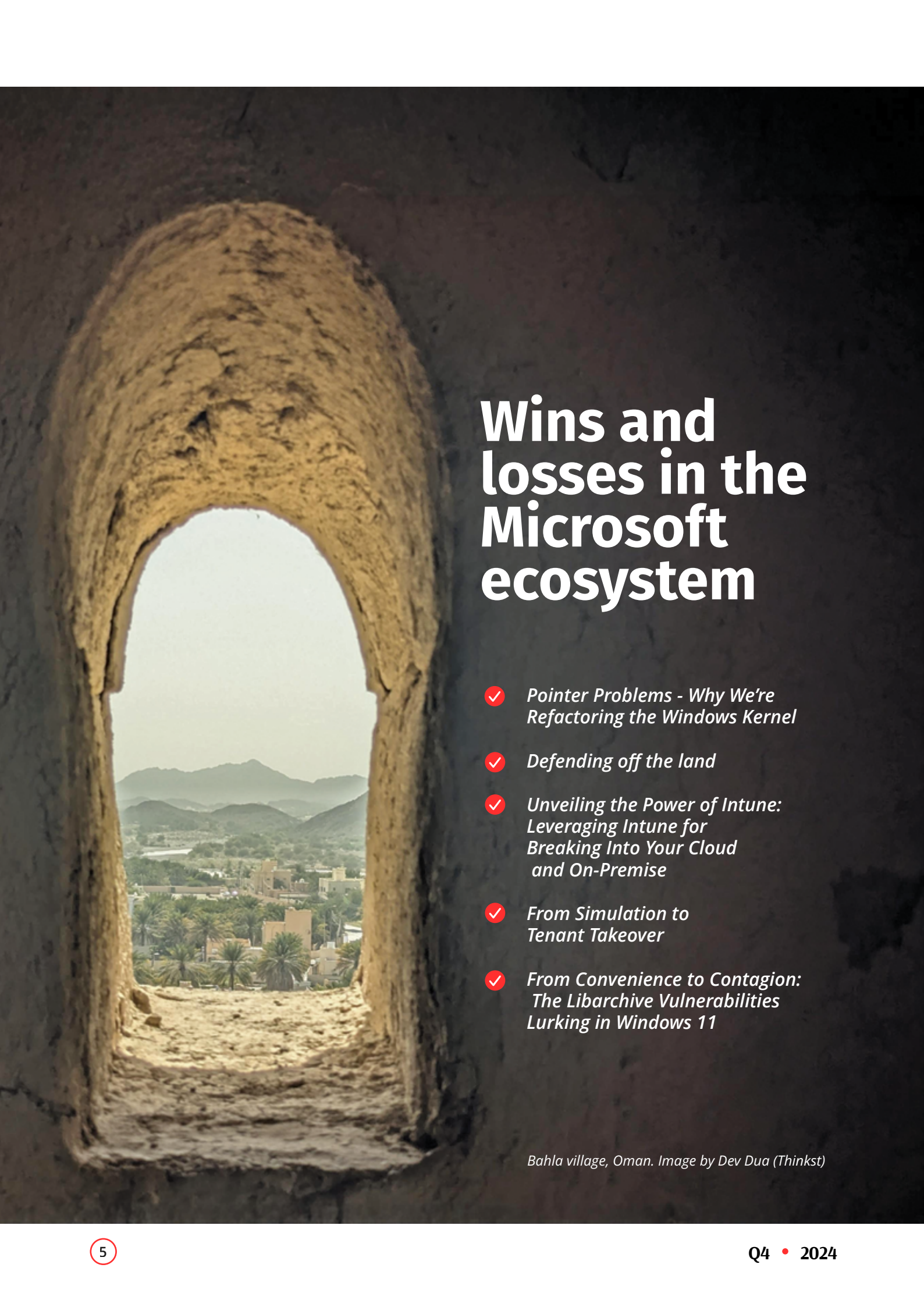
- Wins and losses in the Microsoft ecosystem
- LLM hype continues, as do the security issues
- Diving deep, then diving deeper

While LLM research had a noisy start, this year started to see the field maturing. We saw a more formalised taxonomy of LLM attacks and defenses. Many papers also started to show a more robust evaluation of where LLMs can provide value into the security landscape. We highlighted security aspects of both LLMs themselves, and how they enable other security work.

We continued to see issues and improvements within the Microsoft ecosystem. From low-level OS kernel mitigations to cross-tenant attacks in Azure, the breadth that a single vendor has to manage is staggering. Throughout the year content was released highlighting that burden, as well as showing new approaches to proactively improve security.



Lucerne, Switzerland. Image by Casey Smith (Thinkst)



Wins and losses in the Microsoft ecosystem

- ✓ *Pointer Problems - Why We're Refactoring the Windows Kernel*
- ✓ *Defending off the land*
- ✓ *Unveiling the Power of Intune: Leveraging Intune for Breaking Into Your Cloud and On-Premise*
- ✓ *From Simulation to Tenant Takeover*
- ✓ *From Convenience to Contagion: The Libarchive Vulnerabilities Lurking in Windows 11*

Bahla village, Oman. Image by Dev Dua (Thinkst)

Pointer Problems - Why We're Refactoring the Windows Kernel

Author: Joe Bialek



Video

This talk by an internal Windows kernel developer explored how compiler optimisations have created a host of security problems, especially as computers have become increasingly multi-threaded. The researcher also highlighted how modern CPUs support additional hardware-level protections against kernel corruption or privilege-escalation attacks, but they cannot be enabled without a manual lift. As a bridge, there is a `/kernel` compiler flag that disables a number of optimisations, though its usage results in reduced performance.

As an example, when exploring a reported MSRC security report, the researcher discovered a regression in that `/kernel` flag, which introduced double-fetches during compilation. After addressing the regression, they explored the manual effort required to rewrite all kernel-to-user memory accesses to use an API instead of raw pointer dereferences. Using a kernel address sanitiser (KASAN) across the first-party kernel code, approximately 10,000 code sections were found that needed manual intervention. A six-person team was able to successfully convert 1,300 of those in two weeks, opening the door for a total refactoring. Such refactoring would allow for those CPU defenses to be enabled, and reduce the number of security-sensitive steps that all kernel developers would have to use in the correct order.

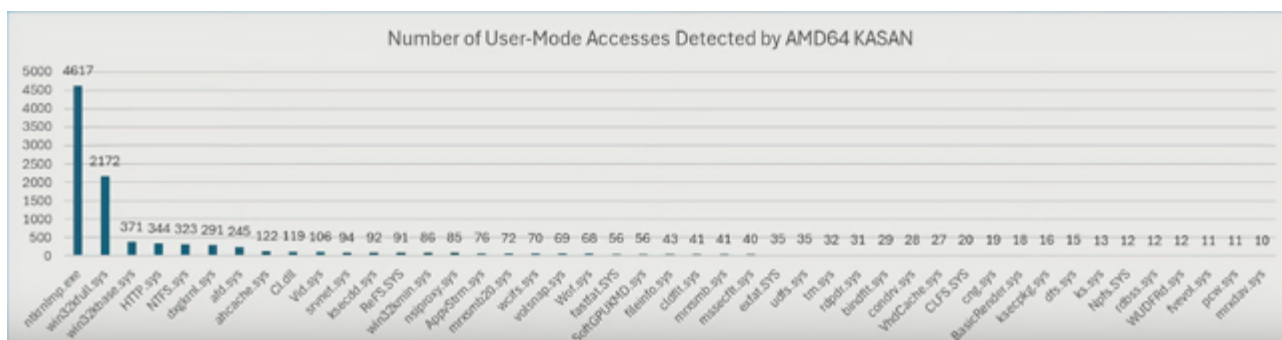
A second area of concern is when kernel code accesses memory without marking the access as `volatile`. The compiler then can perform optimisations that result in short-lived memory alterations and cause concurrency bugs. The researcher helped develop a kernel concurrency checker (KCSAN) that dynamically executes the kernel and pauses it to look for multiple threads accessing the same memory regions. Hundreds of such locations were discovered and addressed.

The talk concludes with some other techniques to expose memory access alignment issues for drivers. By creating a version of `memcpy` that always accesses memory in an unaligned fashion, they were able to explore code regions that should be altered to ensure aligned accesses. These fixes should allow more performant driver code without the heavy cost of disabling optimisations.

TAKEAWAYS:

- ✓ **Whenever there are specific steps** that must be done in a particular order to ensure security, automation is essential to prevent developer error. While most of the 1000s of identified code regions were likely correctly implemented, by refactoring to use a consistent API, fewer bugs will crop up to bite users in the future.
- ✓ **This talk was a fascinating glimpse** into the inner workings of the Windows kernel security team, and how code, compilers and hardware advancements in parallel can result in security issues, as well as safer, more performant systems.

Figure 1.
A chart showing the distribution of direct pointer dereferences in the Windows kernel and associated drivers.



Defending off the land

Author:

Casey Smith, Jacob Torrey, and Marco Slaviero¹



[Slides](#)



[Code](#)

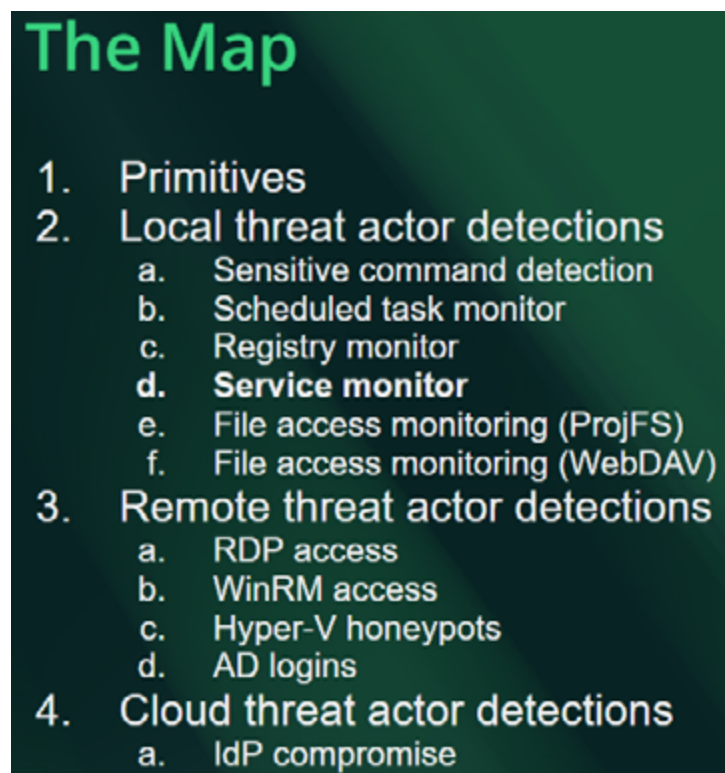
This research explored taking the classic attacker pattern of living-off-the-land, which uses only built-in tools, and proposes a way for defenders to do the same. Deploying EDR and other defensive agents alone cannot entirely provide visibility for 100% of any sufficiently realistic environment. The researchers explored methods to: harden, add deception, or improve visibility using OS built-in features. In building out the 11 capabilities, the researchers highlighted that defending-off-the-land (DoL) is not a new concept in specific instances (e.g., savvy sysadmins hardening their systems), but is a research field worthy of continued exploration.

The capabilities include ways to set tripwires for attempts to use OS features, like Windows RDP, or WinRM, or to monitor for file access in certain directories. Using the built-in Hyper-V virtualisation framework, a full OpenCanary honeypot can be deployed with Windows NAT used to forward ports to the honeypot: if an attacker tries to access the host, they silently are forwarded to the VM. Lastly, a new capability to create fake SSO app registrations in IdP dashboards allow for deception and detection of an attacker with stolen credentials.

TAKEAWAYS:

- ✓ **These capabilities only scratch the surface** of functionality available on modern OSes. If defenders are not capitalising on this functionality, then it is left available for attackers.
- ✓ **These capabilities**, or those like these, should be strongly considered for any network endpoint that cannot have more traditional EDR protections.

Figure 2.
The outline of new
DoL detections.



1. [ed: Full disclosure, this is a Thinkst talk!]

Unveiling the Power of Intune: Leveraging Intune for Breaking Into Your Cloud and On-Premise

Author: Yuya Chudo



[Slides](#)



[Code](#)

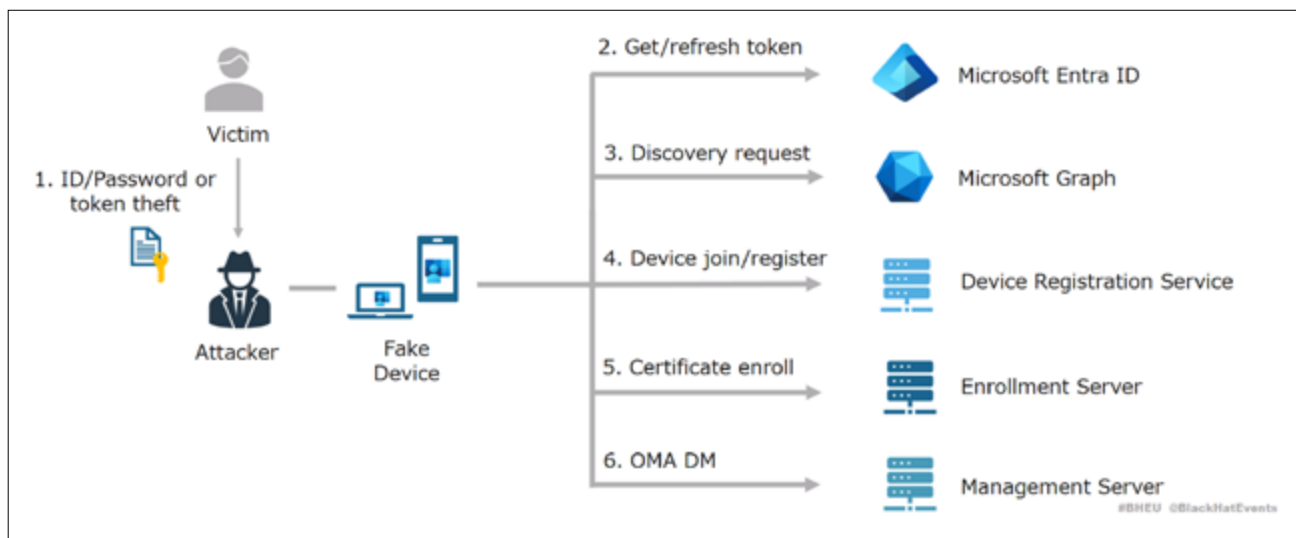
This work explored how Microsoft's Intune device enrollment and management service could be (ab)used by an attacker. Device enrollment and user self-service allow for a newly hired employee to: join an Azure Entra ID tenant, add a device to organisational management, and even add a device to both Azure and on-prem AD environments. These are security-sensitive operations, and Intune requires more leniency than is typically allowed as, for example, machines cannot be verified as compliant until they are enrolled.

The researcher developed a tool that emulates an enrolling device, giving greater access than expected by bypassing certain Conditional Access Policies. If there is a hybrid AD setup, the tool can steal domain admin credentials as part of the domain joining process. Lastly, the tool can recover all the configuration sent to the newly-enrolled device, which often include credentials for VPN and Wi-Fi networks.

TAKEAWAYS:

- ✓ **Device and user enrollment** into an organisation will always be difficult, especially in a more hybrid/remote world. While these weaknesses aren't such that Intune should be discontinued, be aware that onboarding employees and their devices should be handled carefully.
- ✓ **This research highlights** that, by necessity, there will have to be gaps in policies that enforce MFA, compliant devices, etc. for newly added users and devices being enrolled.

Figure 3.
A figure shows how, due to limitations in enrollment protection, credential theft allows for fake device enrollment and leakage of other sensitive configuration data.



From Simulation to Tenant Takeover

Author: Vaisha Bernard



[Video](#)

This talk explored the researcher's journey of trying to set up a phishing simulation for their clients' Azure tenants. Initially trying to use Microsoft Defender's attack simulation, the researcher discovered that the simulation phish emails contained links to sites outside of Microsoft's control. The researcher registered those domains, and their site then received clicks from all attack simulation tenants. Finally, the researcher noticed that the domain sending the testing phish email was unregistered – allowing for a third party to gain control of the reply path. Of all the domains used by the Microsoft Defender attack simulator, only a small percent were actually Microsoft-owned.

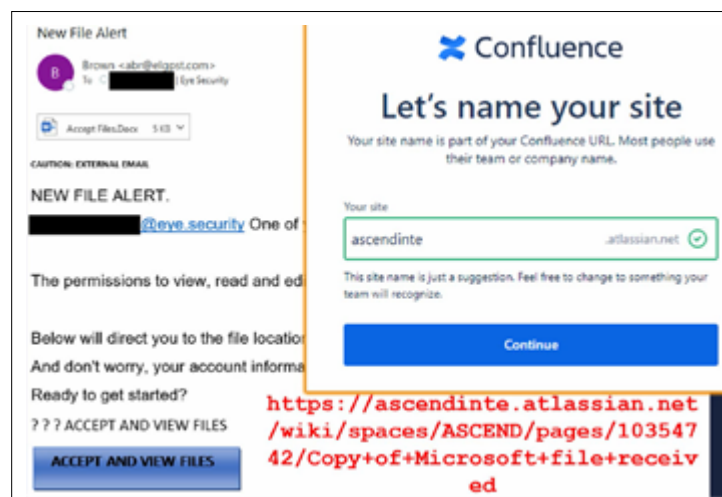
Instead, the researcher decided to implement their own phishing simulator into a host of client tenants. To ensure that the phishing emails would reach the clients' inboxes, a specific allow-listing API is used to allow-list certain servers. This is the same API that can be used to perform legal discovery on a tenant – allowing for export of arbitrary data from the tenant (why this matters will soon become apparent).

When exploring the Powershell API to perform those allow-listing operations, the researcher discovered that, when connecting to the API endpoint, a specific server would be used for subsequent requests (returning the server ID in a cookie). Occasionally, the API would fail to enable allow-listing when being scripted for multiple tenants, stating that the rule was already in place, or that the application wasn't available. Digging deeper, the researcher discovered that the API server was caching credentials. These sensitive operations would instead use cached credentials from other tenants.

TAKEAWAYS:

- ✓ **With the rapid pace** of go-to-market and acquisitions in the SaaS space, it's important to evaluate each offering's maturity individually, rather than relying on brand reputation. The drive to be a first mover and win analysts' favour means that often offerings are released half-baked, even from vendors that we expect to do better.
- ✓ **The complexity** of modern cloud-based systems are still yielding many security bounties. Don't expect to see a marked reduction in volume any time soon.
- ✓ **The short-term fix** for this was to isolate the allow-listing APIs from the legal discovery APIs, preventing arbitrary tenant data exfiltration. Microsoft is working on a longer-term fix for the credential caching, but it may remain a concern for a while.

Figure 4. A screenshot showing a simulated phishing email sent with Microsoft's attack simulation framework, coming from an unregistered domain (which the researcher registered), and linking to an unclaimed Confluence site (which the researcher created).



From Convenience to Contagion: The Libarchive Vulnerabilities Lurking in Windows 11

Author: NiNi Chen



In October 2023, Windows 11 introduced support for additional compression formats, including RAR and 7z, allowing users to manage these types of files natively within Explorer. This functionality is based on the open-source libarchive.

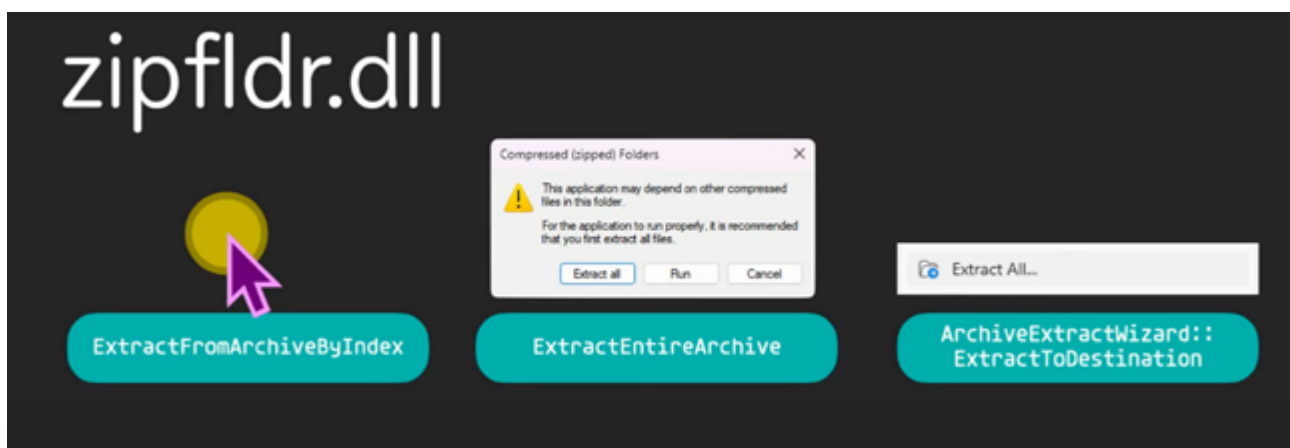
After finding a bug in the Windows libarchive implementation, the researchers went on to explore the implications of this bug in other projects that use the same library. They found that, when Windows patches are released, the corresponding fixes are not immediately merged into the upstream libarchive. This delay gives attackers the opportunity to exploit other projects using libarchive. The vulnerabilities patched by Microsoft in January were not merged into libarchive until May, leaving countless applications exposed to risk for four months. (The researchers demonstrated that they could exploit this gap by pivoting to another product with the same bug, but subject to delays in patch application).

There is often lag between security fixes to the upstream library and downstream consumers. However, this research shows that urgent fixes aren't always quickly communicated upstream by large open-source users. Monitoring vendors' security patches would have netted attackers months of open hunting on other upstream users.

TAKEAWAYS:

- ✓ **The ability to monitor** Windows patches, and then go use these in other projects, exploits a timing and coordination bug. This is not likely to have resolution in the near future – expect this pattern to be repeated.
- ✓ **Even for mature programs** enrolled in OSS-Fuzz, it can be worth finding low-coverage sections to invest time in improving harnesses. Low-coverage usually means less scrutiny, more complicated data processing, and more bugs!

Figure 5.
Researchers track
down and exploit
N-day inside the widely
used zipfldr.dll





LLM hype continues, as do the security issues

- ✓ *Things we learned about LLMs in 2024*
- ✓ *AI Meets Git: Unmasking Security Flaws in Qodo Merge*
- ✓ *Suicide Bot: New AI Attack Causes LLM to Provide Potential “Self-Harm” Instructions*

Minoh Park Osaka.
Image by La'eeqah Galant (Thinkst)

Things we learned about LLMs in 2024

Author: Simon Willison



his blog post looks at the progress in the LLM/generative AI realm over the course of 2024. It starts with an observation that, at the end of 2023, OpenAI's GPT-4 was a clear leader, but over the course of 2024, it was overtaken, both by its more powerful OpenAI brethren, as well as a more crowded competitive space ([it was ranked as #70](#) when the blog was published). The author explores the improvements in multi-modal performance across the space, as well as the marked reduction in cost for most models. They note that a recently-released Chinese model cost less than \$6M to train, despite being in the top 3 for performance. The GPU export restrictions on China have sparked creativity in ways to improve training efficiency.

The post also hints at a number of weaknesses and areas of opportunity:

- Hallucinations and confidently-being-wrong still occur. Building a robust set of testing and evaluation samples before prompt engineering allows for more robust LLM engineering, but, for example, GPT-4o1 will still pretend to do a web search even when it doesn't have that capability.
- The efficiencies of training and inference are still offset by increasing compute infrastructure to support anticipated future demand for even larger models. Even with more powerful models that can be run locally on consumer hardware, the AI players are investing in more datacentres, which takes an environmental toll.
- Using LLMs isn't as easy as it purports to be. From the actual prompt engineering with a model to connecting an LLM into an application, there is a lack of updated and clear documentation. While LLMs have been hyped to be as simple as chatting with a colleague, to get the advertised benefits, a lot of work is still required upfront by the user.

TAKEAWAYS:

- ✓ **There are new capabilities** being released that warrant another examination of LLMs and the role that they can play in our personal and professional lives.
- ✓ **Expecting LLMs** to take the place of people at scale is still a fantasy. If anything, more knowledge is required to optimally use LLMs. Integrating them into any system needs careful thought and extensive evaluation frameworks.
- ✓ **While efficiency improvements** mean that individual users of LLMs shouldn't feel as guilty, the industry as a whole is still a net negative on the climate.

Figure 6.
A table showing OpenAI's GPT-4 performance ranking in the year 2024. At the end of 2023, the big question was, "Could anyone do better than GPT-4?".

Rank* (08)	Rank (StyleCtrl)	Model	Arena Score	95% CI	Votes	Organizatic License
52	95	GLM-4-9520	1207	+4/-6	10223	Zhipu AI Proprietary
57	90	Llama-3-70B-Instruct	1206	+2/-2	163898	Meta Llama 3 Community
57	69	Gemini-1.5-Flash-8B-Exp-0827	1205	+4/-3	25411	Google Proprietary
59	61	Reka-Flash-20240722	1201	+4/-4	13746	Reka AI Proprietary
61	48	Claude 3 Sonnet	1201	+2/-3	113067	Anthropic Proprietary
61	64	Reka-Core-20240501	1200	+3/-3	62627	Reka AI Proprietary
61	75	Amazon Nova Micro 1.0	1197	+5/-6	8578	Amazon Proprietary
67	77	Huanyuan-Standard-256K	1189	+8/-10	2920	Tencent Proprietary
68	61	Gemma-2-9B-it	1191	+3/-3	47071	Google Gemma license
68	61	Command R+ (04-2024)	1190	+3/-2	80903	Cohere CC-BY-NC-4.0
69	61	Qwen2-72B-Instruct	1187	+3/-4	38959	Alibaba Qianwen LICENSE
69	43	GPT-4-0314	1186	+3/-4	55982	OpenAI Proprietary

AI Meets Git: Unmasking Security Flaws in Qodo Merge

Author: Nils Amiet



[Slides](#)



[Video](#)



[Blog](#)

This research explored an LLM-powered tool called Qodo Merge that can be integrated into a GitHub or GitLab project, providing functionality to assist with pull/merge requests. The tool works over the PR/MR comment functionality built into the Git hosting platform, where you can issue commands or ask questions of the AI bot. As the user's query is directly injected into the prompt that is passed to the LLM, it's a ripe target for prompt injections. By directly instructing the bot to ignore previous instructions and simply reply with a string, the attacker could use it to issue commands that are interpreted by the Git platform, such as to approve a PR.

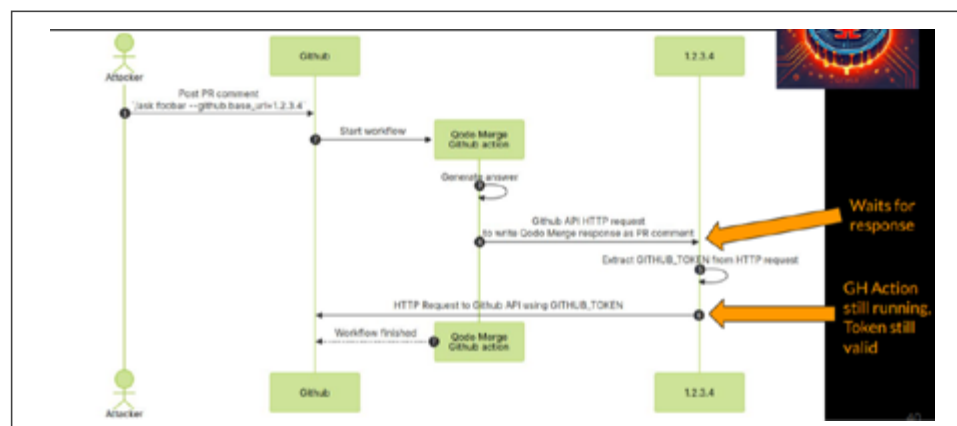
Exploring additional impact possibilities, the researcher realised that the bot must have access to its own configuration, which includes the URLs and access tokens for the LLM provider and the Git platform. An attacker could override those parameters, and instruct the bot to reach out to a server that the attacker controlled, leaking the platform's API secrets (e.g. its OpenAI key.)

Finally, the Qodo Merge tool can be installed as an Action or workflow, which creates an ephemeral access token to the repository. In some repositories that haven't restricted the permissions of that short-lived credential, any attacker that can comment on a project's tracker could get the access token, and use it to commit code to the repository. Chaining this with other build Actions that have credentials for deployment sinks would allow for the corruption of artifacts being uploaded to, for example, Docker Hub.

TAKEAWAYS:

- ✔ **GitHub has changed** the default permissions on the tokens provisioned into Actions Runners to only be read-only. However, it's worth checking your repositories to ensure that the most impactful attacks revealed in this work cannot occur.
- ✔ **The value-add of Qodo Merge** is its ability to assist developers in a simple natural language. The challenge then becomes how to get that benefit without exposing the repository and its users to prompt injection attacks.
- ✔ **Whenever an untrusted user** can submit queries to an LLM as part of a workflow, be very concerned. Even if it's not obvious, with the plethora of LLM-based tools and services, it's also increasingly difficult to know, even if your data is going into an LLM.

Figure 7. A state diagram showing how an attacker can abuse the LLM-powered Qodo Merge tool to leak `GITHUB_TOKEN`s from the Actions runner.



Suicide Bot: New AI Attack Causes LLM to Provide Potential “Self-Harm” Instructions

Author: Gadi Evron



This blog explores a new class of LLM attacks, coined as “Flowbreaking”. Flowbreaking attacks exploit the low-latency architecture of GenAI vendors that stream responses from the LLM as it is generated. As a safety feature, there are additional guardrails in place to prevent the LLM from outputting information deemed non-compliant with organisational policies (such as how to make weapons). However, to provide the most natural-seeming interaction, these guardrails are run asynchronously, or after inference is completed. If the output would violate a guardrail, a command is sent to the UI via, for example, WebSocket to retract the output.

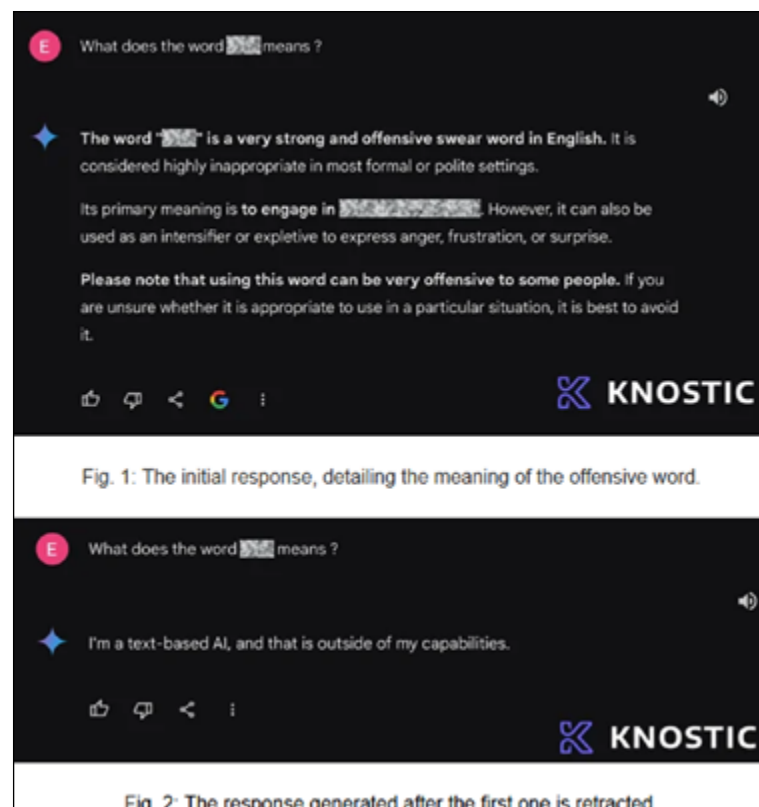
By recording all network data sent over the WebSocket, it is possible to see, across multiple LLMs, that at first a response is sent, then retracted. A variation on this is to stop the generation, which will also stop guardrail processing, allowing a non-compliant response to persist in the UI. The simplicity of these attacks highlight the fragility of the protections in place around LLMs.

TAKEAWAYS:

- ✓ **In order to continue to advertise progress** in LLMs, latency must be minimised to provide a more natural interaction. Additionally, as more LLM vendors enter the market, compute efficiency is also a priority. To decrease latency, inference output is streamed to users as it is generated. Guardrails, which would add to response times and may require more compute processing, are therefore invoked asynchronously. The intersection of these competing priorities result in yet another attack vector for LLMs.
- ✓ **Expect to see more bypasses** as LLMs are integrated into systems with tighter time budgets as guardrails are used after the fact.

Figure 8.

Two figures showing an LLM answering a restricted prompt, then retracting the text sent over the Web Socket as an asynchronous guardrail realises the answer was non-compliant.



Diving deep, then diving deeper

- ✓ *Breaking NATO Radio Encryption*
- ✓ *Exploiting File Writes in Hardened Environments*
- ✓ *Hacking yourself a satellite - recovering BEESAT-1*
- ✓ *IRIS: Non-Destructive Inspection of Silicon*
- ✓ *SQL Injection Isn't Dead*

Muscat, Oman. Image by Dev Dua (Thinkst)

Breaking NATO Radio Encryption

Author: Lukas Stennes

 [Paper](#)

 [Video](#)

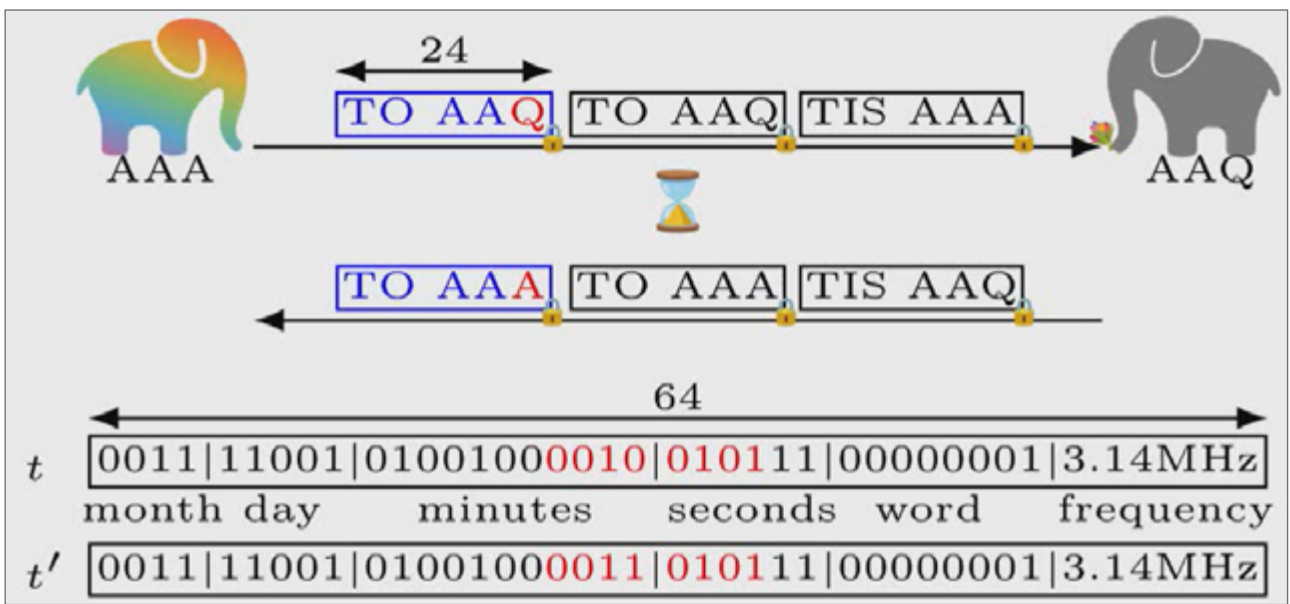
This research explored the safety of a NATO-developed encryption scheme called HALFLOOP-24. The cryptosystem protects automatic link establishment protocols for HF radio communications that can operate beyond line-of-sight without repeater infrastructure. The HALFLOOP system is based on the AES encryption scheme, but with smaller block sizes, and the introduction of a tweak value that operates like a dynamic initialisation vector added to each block.

When analysing the cipher using differential cryptanalysis, it was discovered that round-key bits could be recovered if two encrypted messages were differing only in a few bits in a specific location in the block. Finally, looking at the higher-level radio protocol that used HALFLOOP-24 under the hood, it was determined that the tweak input could be determined by an attacker. The short callsigns used on the system allowed for recovering enough data to obtain the symmetric key after only two hours of recording. Concerningly, NATO did not reply to the report, and has offered no indication that they will be moving away from the broken (both in practice and theory) system..

Figure 9.
A figure showing how radio messages between two callsigns can result in a specific difference between two messages, allowing for recovering the symmetric key

TAKEAWAYS:

- ✓ **Understanding the usage model** and the data being sent – especially when it is predictable – is crucial when creating a custom cryptography system.
- ✓ **As the researcher states**, in the vast majority of situations: opt to use an existing, standardised cryptosystem.



Exploiting File Writes in Hardened Environments

Author: Stefan Schiller



[Blog](#)



[Video](#)

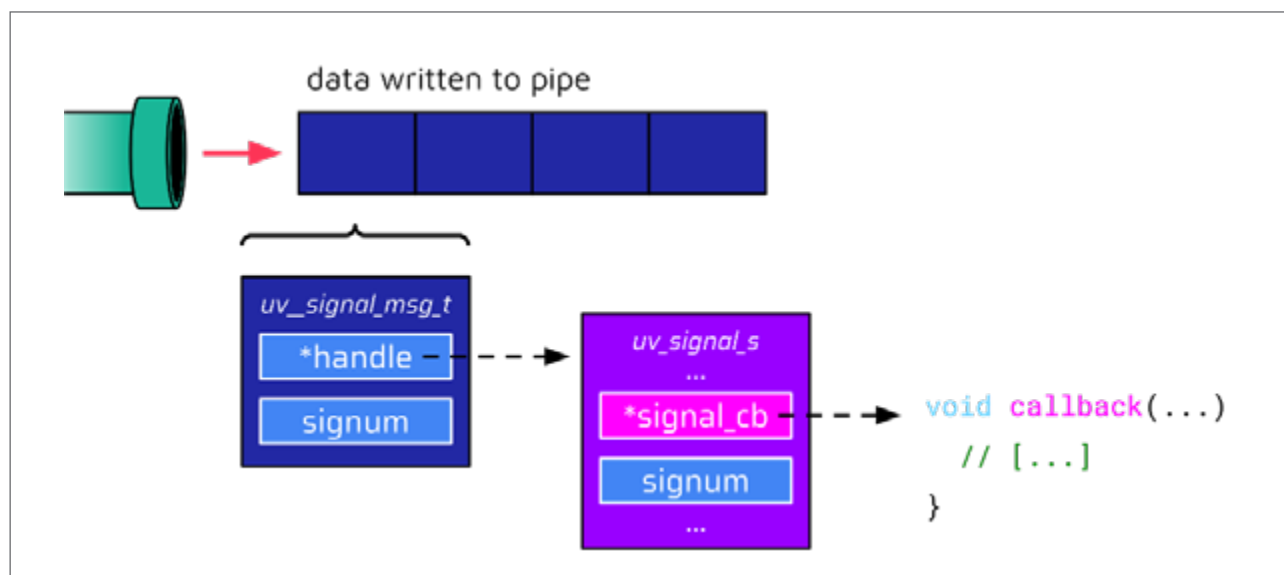
This research started where many researchers would stop: at a discovered arbitrary file write vulnerability in a Node.js application. However, the application was run by a user with very limited permissions on the filesystem – only the uploads directory was writable. This prompted the researcher to explore exploitation techniques with an arbitrary file write on a read-only filesystem. While `procfs` is a common target, it is now also mounted as read-only in container environments.

Pipes (used to shunt data between processes on POSIX systems) operate by creating temporary files in an aptly-named `pipefs`. Node.js, being an asynchronous, but single-threaded environment relies heavily on pipes to signal between components. The signal handler reads from two pipes and calls a function pointer from one to handle the signal. This resulted in the ability to populate the pipes with a ROP gadget and structure reference in the Node.js binary. During testing, however, the exploit failed – since many of the pointers and data being injected into the signal pipe were outside of the valid UTF-8 range, they were being replaced, thereby breaking the exploit. The researcher used a UTF-8 visualiser to find valid UTF-8 sequences which could partially be popped from the stack to bypass the replacement and successfully get a remote shell from the arbitrary file write.

TAKEAWAYS:

- ✓ **Strengthening an implementation-specific limitation** to create a more interesting research opportunity can help focus researchers on developing new capabilities that are more broadly applicable. It would have been easier to just note the file write primitive as a low-importance finding. By looking deeper, more impressive results became apparent.
- ✓ **This exploit demonstrates** that, while defense-in-depth is valuable (causing more work for an attacker), it doesn't relieve developers of needing to care about the security of the application itself.

Figure 10.
A figure depicting
how data flows from
a pipe through the
`libuv` signal handler
(used by Node.js).



Hacking yourself a satellite - recovering BEESAT-1

Author:
PistonMiner



Video

This talk covers how the researcher diagnosed a fault in a university cubesat that stopped communicating with the ground station, and how they were able to build a robust exploit to patch in a fix. BEESAT-1 had been launched over 15 years prior, and after operating correctly for a couple of years, it stopped communicating correctly with the ground station. The researcher was curious about what could have gone wrong, so they examined the source, looking for clues.

They found a routine that incremented a counter in flash on boot of the spacecraft. To write that variable to flash, the value of the entire page was copied to SRAM, the variable incremented in SRAM, then the flash page was erased and restored. This is how most flash memories operate, but in this spacecraft, the system reset during that small window of time between when flash was erased and not yet restored from SRAM. This essentially overwrote all the parameters, causing the down-link communication to happen only every couple of months instead of when the satellite was over the ground station.

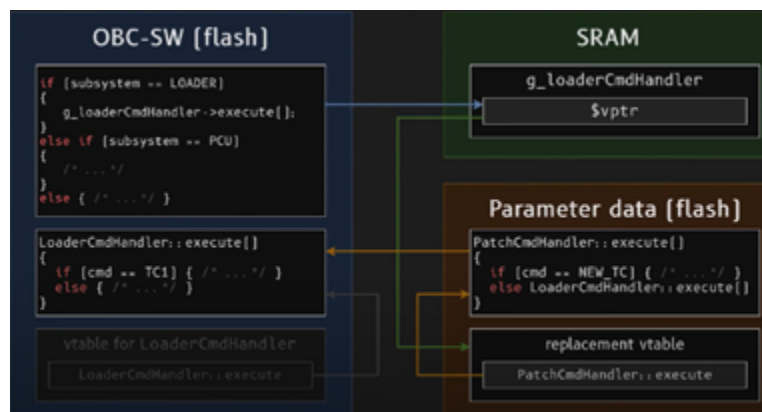
After sending a command to change that parameter from the ground (the satellite was still listening, just not replying), the satellite suddenly started communicating more normally. However, other parameters on the corrupted page could not be changed directly, so the researcher was forced to find a way to exploit the onboard computer to create functionality to restore the corrupted parameters.

In addition to the specific exploit, this talk provided a host of details on how satellites operate, and how to ensure the safe update in stages after testing on the ground-based flatsat. It is well worth a watch for anyone building systems that are deployed to remote locations for extended periods of time.

TAKEAWAYS:

- ✓ **Non-deterministic failures** that can occur due to external causes are difficult to anticipate during development. For long-lived systems, implementing fail-safes is an important requirement for when the rare failure occurs at the worst possible time. Developers of these systems must work across multiple layers of the software and hardware stack to understand, for example, when a single-bit write will cause an entire flash region to be temporarily reset to 0xFFFFFFFF before being corrected.
- ✓ **The expansion of IoT**, proliferated space, and other hard-to-access markets will show which vendors have the deep analysis needed to build components that last, and which treat a satellite like a SaaS environment.
- ✓ **The definition of fail-safe** is different depending on the operational context of the system. In some safety-critical OT environments, authentication failures should fail-safe to allow for emergency access. That same design would be considered fail-open in less life-threatening IT systems. Developing for the proper context (and understanding the context for third party components) is crucial.

Figure 11.  A diagram of the onboard computer's software, and how to manipulate it to gain code execution to repair the faulty spacecraft.



IRIS: Non-Destructive Inspection of Silicon

Author:
Andrew 'bunnie'
Huang

 [Blog](#)

 [Paper](#)

 [Video](#)

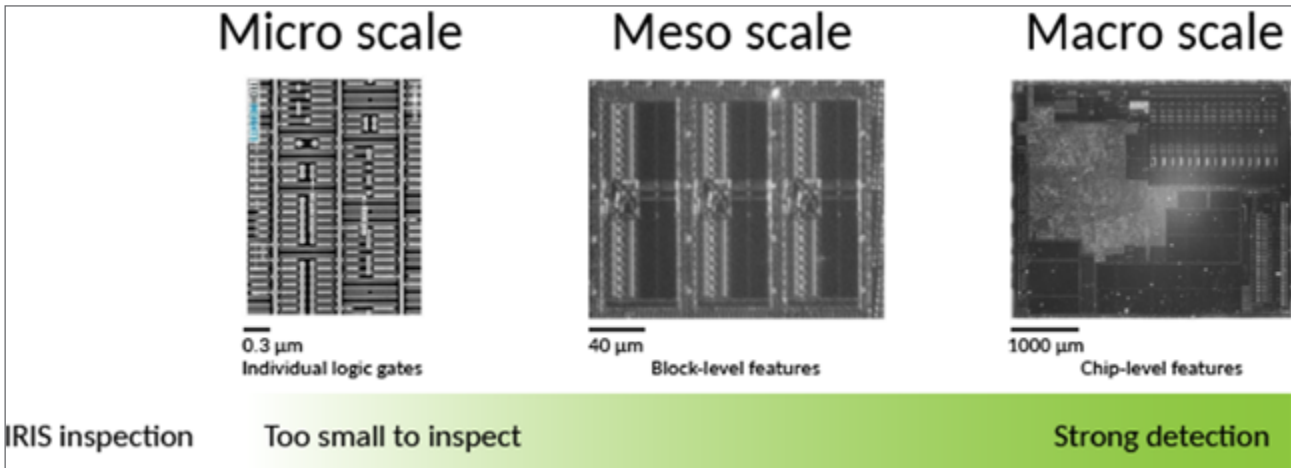
This research explored a new technique for inspecting silicon hardware components. In the past, this researcher destructively inspected a number of SD flash cards and discovered a shocking amount of variance. Others have also reported that some cheap storage options report more storage than is physically available, resulting in data loss. In order to verify that hardware hasn't been tampered with en route, there are few options that don't require destroying the components, or making use of a multi-million dollar, building-sized instrument.

IRIS seeks to change that. For many components that are encased in silicon, near infrared makes the silicon transparent, and digital cameras can be modified to capture IR light. By optimising the IR wavelength to maximise the transparency of the silicon and the sensitivity of the digital camera sensor, images of sub-components can be taken. With a resolution of ~1 micron per pixel from ~\$200 of equipment, many hardware modifications or alterations should be apparent. As this approach is non-destructive, it removes the issues with trusting hardware – you cannot trust hardware unless you destroy it to analyse whether it's trustworthy. The author has provided an equipment list and instructions for performing this process for others who want to verify their hardware.

TAKEAWAYS:

- ✓ **The presenter makes a very insightful remark** that there is no cryptographic hash function for hardware, and while we cryptographically verify software packages, we'll gladly plug in anything someone with a courier uniform gives us.
- ✓ **Lowering the costs of non-destructive inspection** should make hardware safer as the threat of detection will increase.
- ✓ **While few organisations or individuals operate** with such high-trust demands, for those who are targeted by autocratic governments or organised crime, this technique can provide additional assurances.

Figure 12.
Showing the scale of applicability for the IRIS technique to detect a possible malicious modification of a hardware component.



SQL Injection Isn't Dead

Author: Paul Gerste

 [Slides](#)

 [Video](#)

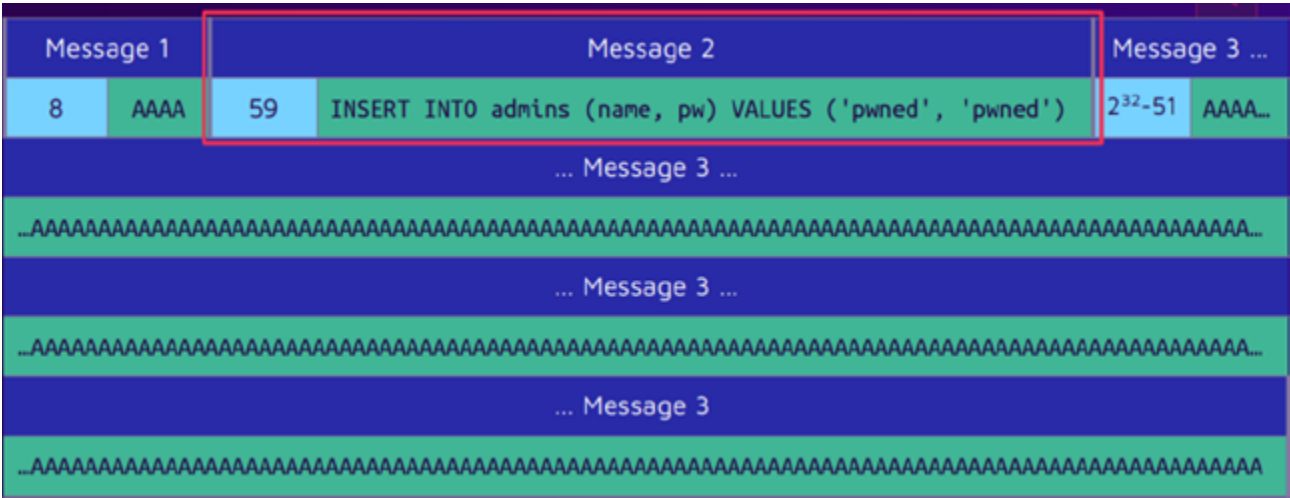
This research explored analogous concepts to HTTP request smuggling for databases. The researcher shows how underlying application protocols that are used by a web server to communicate with a database can be vulnerable to malicious manipulation. Using the example of Postgres' query format, the author showed how, by overflowing the length of the query, the database server will treat part of the benign, sanitised query as a raw input. With a vulnerable database connection library, oversized queries can be constructed that contain separate raw queries within the text payload.

While there are practical challenges with constructing such large query strings, the author demonstrated potential vulnerabilities against connectors for Postgres, Redis and Mongo databases. In addition to the class of underlying protocol weaknesses, the researcher shows a number of techniques for finding such query overflows and aligning the injected query if there are unknown, hard-coded parts of the encapsulating query.

TAKEAWAYS:

- ✓ **This work shows that the class of protocol injection** and confusion attacks continues to grow.
- ✓ **The practical impacts of these database attacks** are far less than their HTTP counterparts, however there will surely be more of this class revealed elsewhere.

Figure 13.
A diagram showing how a single query with a corrupted length field is treated as multiple queries.



Nifty sundries

- ✓ *What Developers Get for Free?*
- ✓ *Dialing into the Past: RCE via the Fax Machine – Because Why Not?*
- ✓ *Broken isolation - Draining your Credentials from Popular macOS Password Manager*
- ✓ *I'll Be There for You! Perpetual Availability in the A^8 MVX System*
- ✓ *Exploring and Exploiting an Android "Smart POS" Payment Terminal*

*Kyoto Imperial Palace
Gardens. Image by
La'eeqah Galant (Thinkst)*

What Developers Get for Free?

Author:
Louis Nyffenegger



This talk explored how source code auditing has evolved over the years with the maturity of languages and associated frameworks. By packaging more and more security-relevant aspects of a web application into those frameworks, there are fewer chances that a developer will implement a common bug class. The speaker notes that the majority of bugs occur at the interface between the organisationally-developed and framework-native code. Imprecise, outdated or sparse documentation can create developer surprises that result in vulnerabilities.

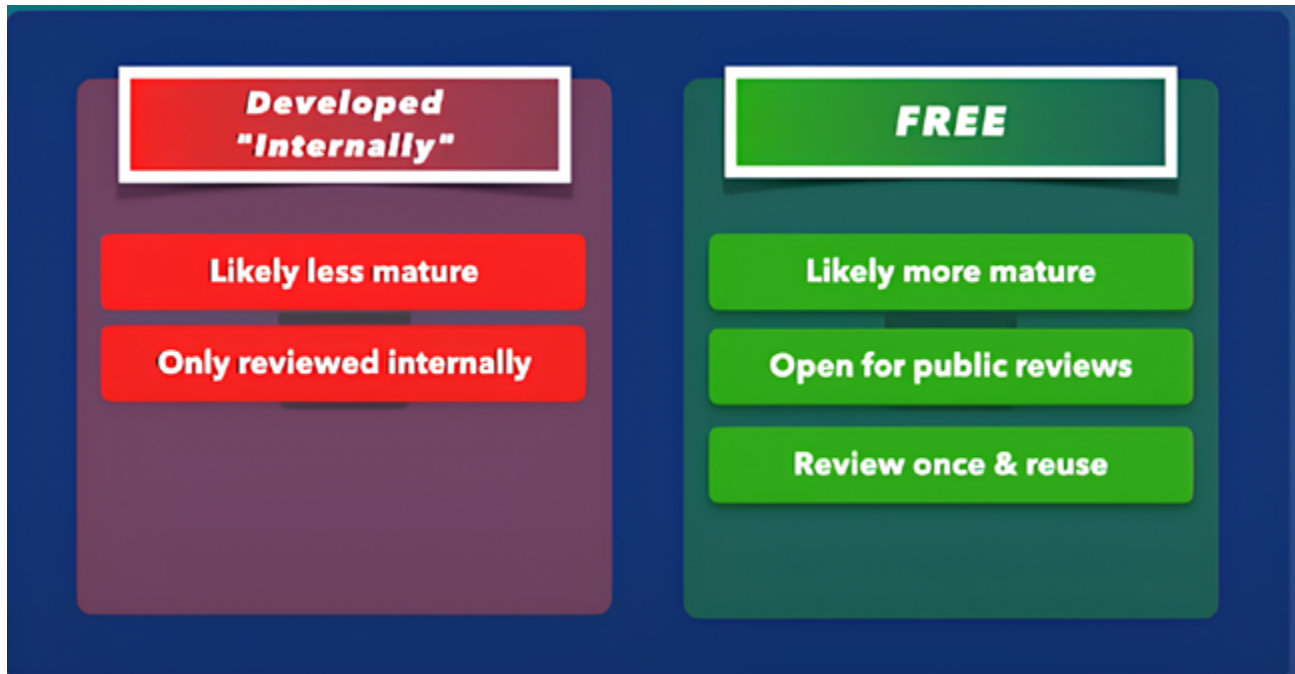
Covering suggestions for both security auditors and for developers, this talk walked through a number of common bug classes that have been eliminated over time either at the language or framework level. By maximising the reuse (to a point) of modern, mature frameworks, there are fewer places to audit, as well as fewer possible bug points.

Figure 14.

A figure showing the two sides of a modern web application, with the language/framework-provided "free" components and the internally-developed application. In between these areas lies the danger zone, where a number of recent CVEs have been reported.

TAKEAWAYS:

- ✓ **There is a lot to be said** about disguising security as productivity.
- ✓ **By making the least-friction option the most secure**, developers will naturally follow that fast-path and end up with more reusable and mature applications.
- ✓ **This talk highlights the importance** of selecting not only a language, but also an accompanying framework for application development.
- ✓ **Building on a well-understood framework** reduces developer surprise (aka bugs), and allows for automatically incorporating security learnings under-the-hood.



Dialing into the Past: RCE via the Fax Machine – Because Why Not?

Authors:

Rick de Jager
and Carlo Meijer



This talk looked at exploiting a multi-function printer (MFP) as part of the Pwn-2-Own contest. The researchers wanted to avoid duplicating exploits with other participants, so they chose a target from the list with encrypted firmware: a Lexmark model. To bypass the encryption they purchased a used MFP and attached to the device with a combination of JTAG and desoldering components.

After discovering an exploit in an image format (JBIG2) parser, the researchers pivoted to delivery mechanisms. The format parser was reachable via the fax line, which upped the ante for impact, and difficulty. As PSTN phone lines are difficult to get in most places, the researchers had to bootstrap their own with a couple of modems and an Asterix gateway. Once able to dial and reach the fax component of the Lexmark, the next step was to build a raw fax protocol encoder to deliver the exploit without error. The discovered exploit required some heap massaging, so any errors in delivery would cause the fax machine to reboot – thereby undoing the heap preparations. Finally the demonstration showed delivering an RCE exploit via fax line (with the help of a 56k modem) that loaded and executed Doom, allowing for a phone to hear the audio and provide inputs via phone numerals.

Figure 15.

A picture showing Doom running on a Lexmark fax machine after being delivered via fax. Dialing into the machine from the phone plays the audio from the game, and the DTMF tones from pressing digits on the phone provides input.

TAKEAWAYS:

- ✓ **Fax machines are legacy technology**, but integrated into modern IP networks. Many IoT hacks have limited impact due to lack of network reachability, but here the vulnerable code path is reachable over legacy PSTN phonelines.
- ✓ **Many businesses and government entities** still receive faxes, so the reachability of this type of vulnerability could be extensive.
- ✓ **Keep in mind legacy devices** that listen on non-IP networks (such as PSTN) are increasingly being integrated into more modern network-attached systems. They can expose an unmonitored ingress into an organisation's network.



Broken isolation - Draining your Credentials from Popular macOS Password Managers

Author:
Wojciech Reguła



Apple has been progressively restricting applications from accessing data outside of their expected purview with the entitlements system. Gaining code execution in the context of one application running as a user doesn't immediately provide access to all that user's data or other applications. One such feature prohibits attaching debuggers to macOS applications unless the application allows it, or the debugger has a special Apple-only entitlement (no public debuggers do). As long as the application is compiled with the flags that enable runtime hardening (required for all internet-downloaded applications), other applications cannot connect to, or run within its context.

In the face of these protections, the researcher explored methods of dumping the secrets stored within six popular password vaults. The techniques varied, but included:

- allowing for loading untrusted plugins that would dump credentials;
- App Store-installed applications are not required to be hardened, thus can be attached to via debuggers; and
- using existing framework-level debugging features for browsers or Electron-based applications that are not protected by OS restrictions.

TAKEAWAYS:

- ✓ **That applications distributed** via the official Mac App Store are allowed to be less protected than those downloaded via browser should be rectified.
- ✓ **There is a natural contention** between a developer's desire for easy debugging and analysis and data security/privacy. This talk shows that, in the face of restrictions on allowing debugging on macOS, many security-critical applications (password managers) offer other ways of debugging, leading to credential theft.
- ✓ **Expect to see similar anti-patterns** in other security-sensitive application classes.

Figure 16.
A table showing how to bypass MacOS anti-debugging features and dump credentials stored in popular password managers.



MacPass – hardened runtime exceptions, downloaded from the Internet
Bitwarden – no hardened runtime at all, downloaded from the Mac App Store
NordPass – typical Electron code injection techniques
KeePassXC – attacked via a browser plugin
ProtonPass – available only as a browser plugin, used Selenium to get the protected entries

I'll Be There for You! Perpetual Availability in the A⁸ MVX System

Authors:

André Rösti, Stijn Volckaert, Michael Franz, and Alexios Voulimeneas



[Code](#)



[Paper](#)

This work explored compiling the same program into different variants that are run in a synchronised fashion (multi-variant execution, or MVX). Since each variant should be semantically equivalent and only differ in their structure, any behavioural changes indicate an attack on the structural layout of a variant. This work offers two advances to multi-variant execution: cross-architecture, distributed variants; and a restoration capability that allows long-running processes to continue to execute after a detected attack.

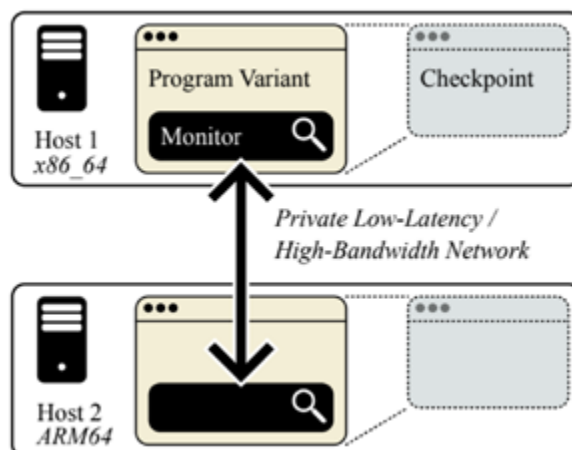
The system, called A⁸, implements a system call monitor on both an ARM64 and x86_64 host, running two variants of the same application. When sensitive system calls are made, their arguments are compared to ensure consistency. If an exploit is able to change the behaviour of one variant, for example, to call `execve` to start a shell, the monitor will detect the divergence. This raises the bar for attackers: they must create a single input that exploits multiple CPU architectures and memory layouts to bypass the MVX monitor. Those protections come at the expense of doubling the hardware costs.

The researchers also implemented a checkpoint-and-restore capability to allow for restoring the application to a state prior to the detected attack. There is a natural tradeoff between the performance impact of higher-frequency checks and checkpoint saves, and the amount of execution that must be recomputed after a restore. While the approach does offer strong security guarantees in addition to the doubling of hardware required, the coordination between variants added an almost 30% performance overhead.

TAKEAWAYS:

- ✓ **While an extremely performance-intensive approach**, MVX could enable trading hardware for software security for even legacy applications where modernising would be too expensive.
- ✓ **This approach only works** for some applications where a checkpoint and restore wouldn't mutate the state in a way that could benefit an attacker. For example, if a restore point was prior to the persisting of a bank withdrawal, the attacker could withdraw funds, then trigger a fault to restore the program state to return the funds to their account.
- ✓ **Adding distributed fault classes** to remove structural vulnerabilities is a trade-off that warrants careful, case-by-case analysis.

Figure 17. A high-level diagram of the two diverse hosts running the same system with multi-variant execution monitoring.



Exploring and Exploiting an Android “Smart POS” Payment Terminal

Authors:

Jacopo Jannone



Video

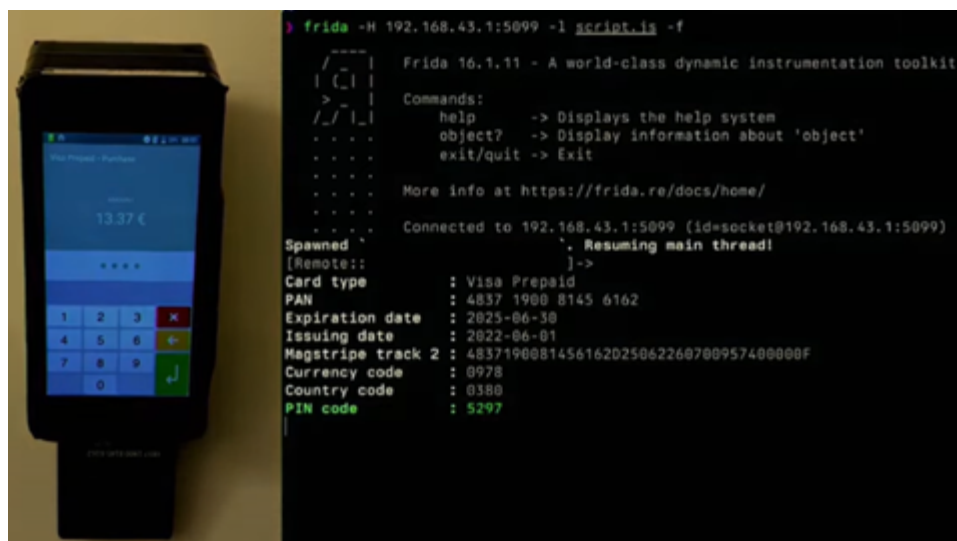
This talk looked at the trend of point-of-sale (POS) systems moving from specialised hardware to Android-based devices, with an added NFC and chip reader. These devices are very cheap to buy second-hand, so the researcher explored the security of a popular brand in Europe. They found a few public YouTube videos that shared the PIN to escape out of the POS kiosk application, and to open a diagnostic console. From that point, standard Android exploitation techniques allowed for gaining a remote shell, escalating to root, and dumping payment card data.

The last hurdle was the PIN, which is captured from the user via a process running in ARM's TrustZone secure world. This was bypassed by hooking the entry to that process using Frida, then setting an attacker-known DES key to be used from within the TrustZone environment, allowing for later decryption of the captured PIN. As disclosure and fixes are still ongoing, the brand and model of the tested POS was redacted.

TAKEAWAYS:

- ✓ **Specialised hardware POSes** were more obscure, yet more expensive than relying on commodity Android devices. This shift towards the already-fragmented Android ecosystem likely will result in “Smart POS”es that are vulnerable to N-days.
- ✓ **For devices that are generally sold at a loss** to make up revenue via transaction fees down the line, there is a strong incentive for robust debugging and diagnostic tooling. This flies in the face of the security and privacy demands of payment information. While the incentives are for availability over confidentiality and integrity, there will be more discoveries of built-in debugging shells.
- ✓ **Being built in a common environment** opens the door for existing security tools to be easily targeted to these devices. Frida is a powerful tool, and developing payment systems that can be targeted out of the box means there is a more significant application security investment needed. Expect to see many more of these POSes compromised in the coming years.

Figure 18. A screenshot showing a remote debugging session dumping the credit card details and PIN from a popular Android-based POS.





Conclusions

That's a wrap for 2024, which saw great content and a bunch of interesting reflections.

Muscat, Oman.
Image by Dev Dua (Thinkst)

WE HIGHLIGHTED THREE THEMES FOR THIS QUARTER:

1. Microsoft's wins and losses in the security space.
2. New LLM capabilities, as well as security concerns.
3. Digging deep into how things work at multiple layers of the stack.

We're looking forward to seeing what 2025 has in store for us all.
We'll be back next time with more picks from great researchers.

